Intro a Ruby

En donde vamos a ver una introducción al lenguaje Ruby, un repaso de subclasificación, y una motivación para mixines.

Universidad Nacional de Quilmes

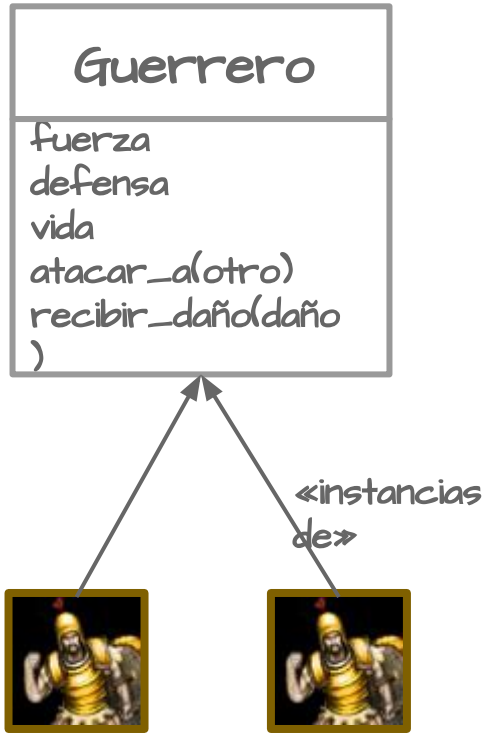
Fuente: steamcommunity.com/sharedfiles/filedetails/?id=400496557

Guerreros 🪓

Se nos pide desarrollar un juego de estrategia, donde tenemos unidades (guerreros) que pueden atacar a otros guerreros y ser atacados.

Cuando un Guerrero ataca, ocasiona un daño a la otra unidad igual a su fuerza menos la defensa del atacado. Al dañarse una unidad, disminuirá su vida.

La solución hasta ahora



Espadachines 🤺

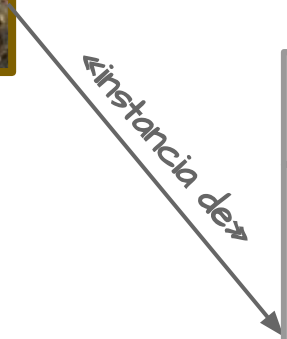
Los espadachines son iguales que los guerreros, pero además tienen (y usan) una espada. Esto modifica su ataque, multiplicándose el mismo por un coeficiente de ataque que depende del arma que tengan.

¿Qué es lo que necesito?📜

Vs.

- ✅ fuerza
- ✅ defensa
- ✅ vida
- ✅ atacar_a(otro)
- ✅ recibir_daño(daño)
- ✏️ ataque

Copiar y pegar como técnica de reuso de código



Guerrero

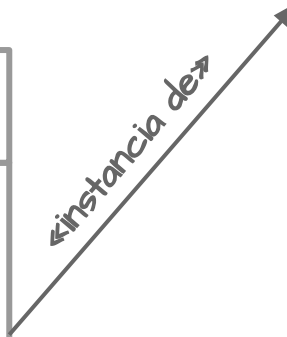
fuerza
defensa
vida
atacar_a(otro)
recibir_daño(daño)

ataque

Espadachín

fuerza
defensa
vida
atacar_a(otro)
recibir_daño(daño)

ataque

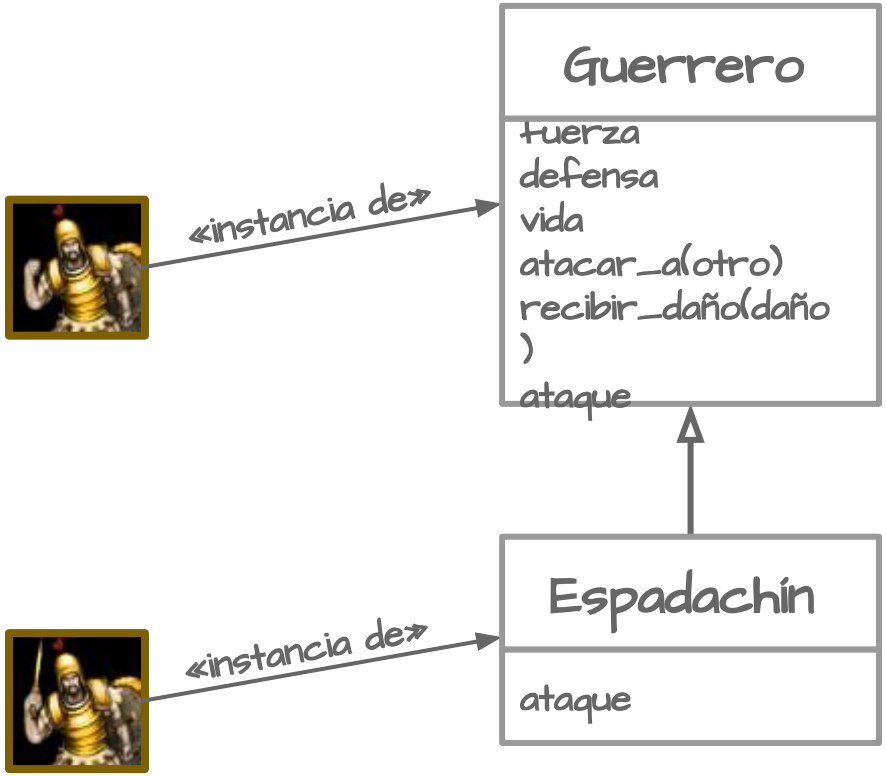


Revisemos el problema 🔍

Vs.

- ✅ fuerza
- ✅ defensa
- ✅ vida
- ✅ atacar_a(otro)
- ✅ recibir_daño(daño)
- ✏️ ataque

Solución usando herencia ✨



Herencia == Reusar código

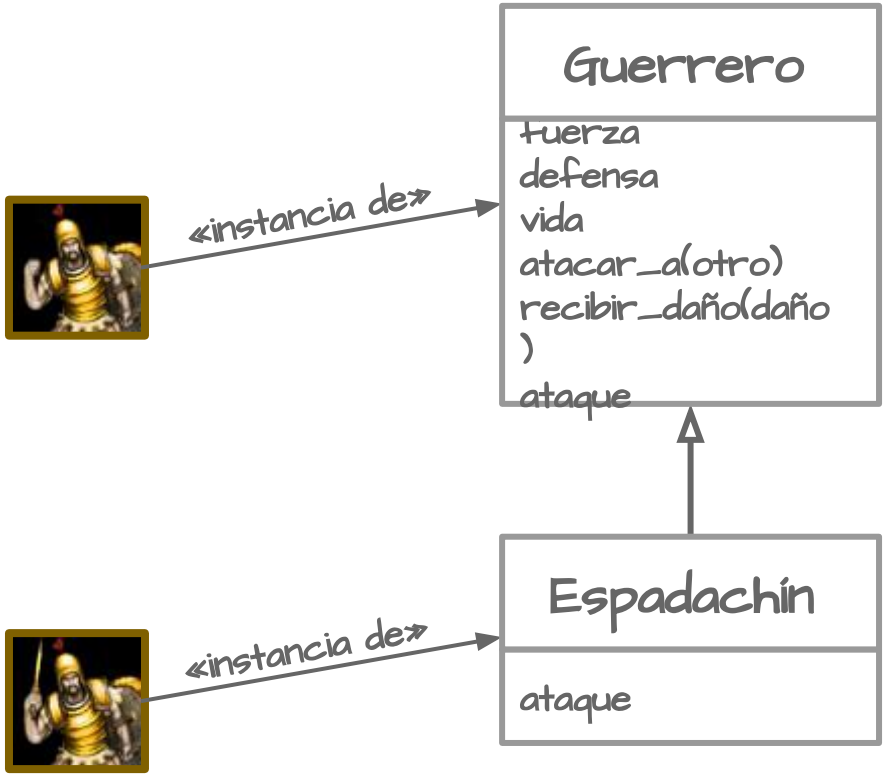
¿Herencia == Reusar código? 🤔

Revisemos el problema 🔍

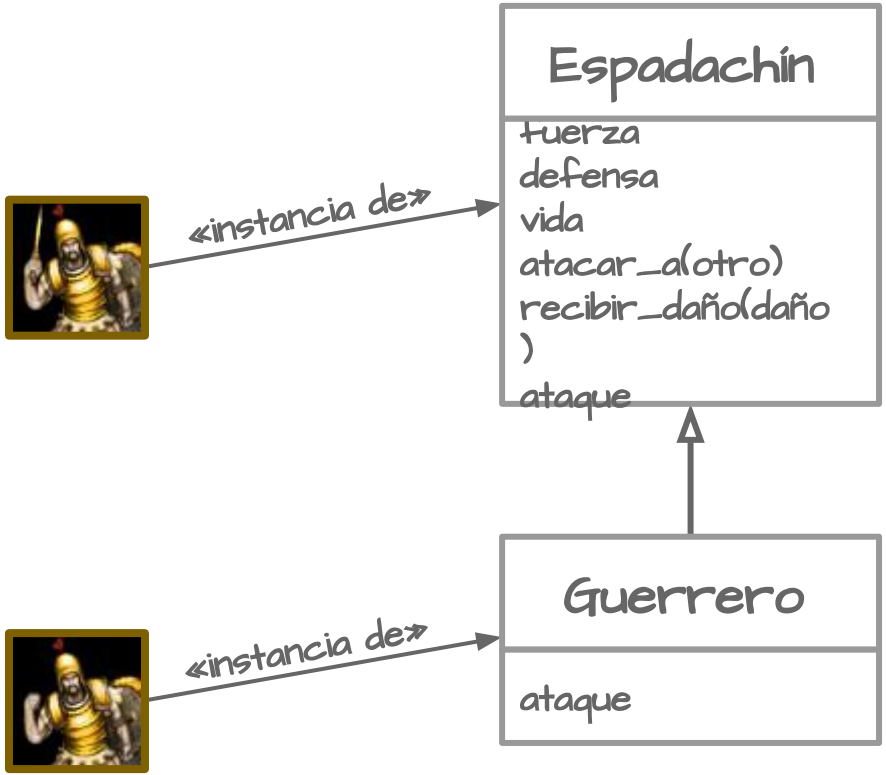
Vs.

- ✅ fuerza
- ✅ defensa
- ✅ vida
- ✅ atacar_a(otro)
- ✅ recibir_daño(daño)
- ✏️ ataque

La solución hasta ahora ✨



Una solución alternativa 🔮



Herencia

- Me permite reusar comportamiento e interfaz dinámicamente, dado el mecanismo del method lookup (parte “mecánica”).

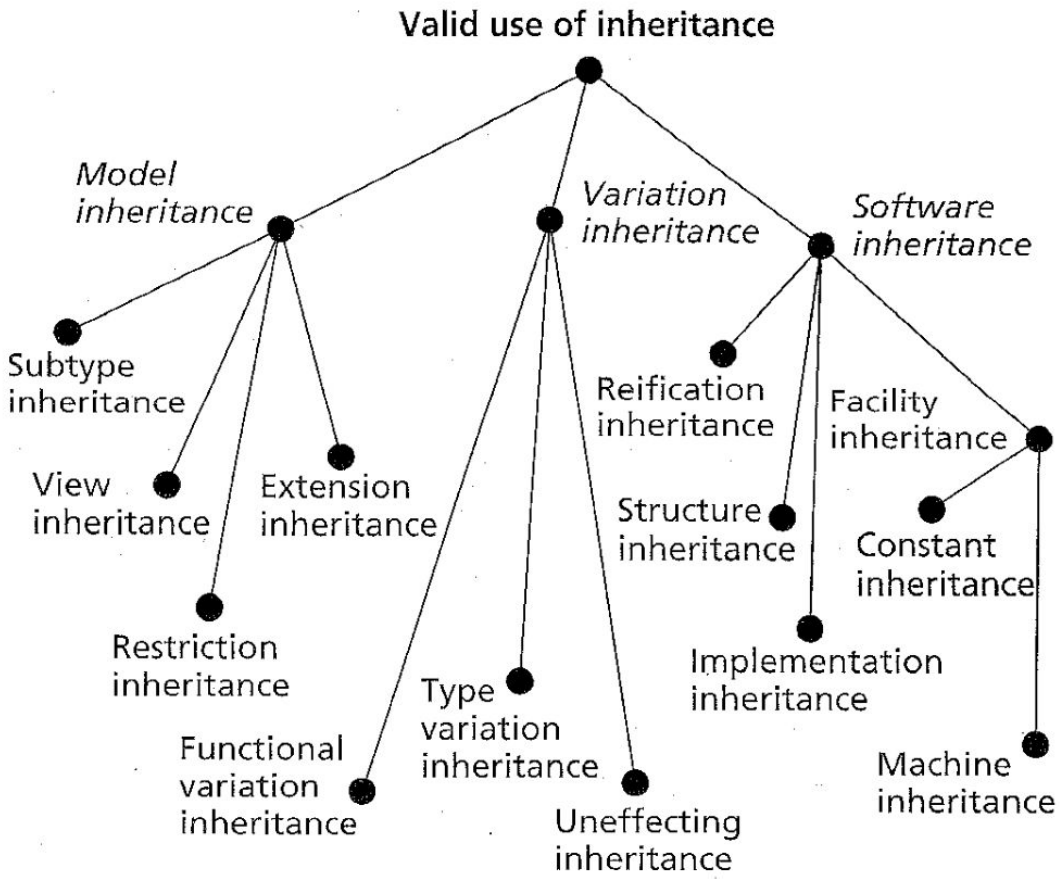
- ⚠️ Representa una teoría del conocimiento de una parte del dominio del problema. Sólo subclasificamos si es válido en el dominio.

Herencia

- Me permite reusar comportamiento e interfaz dinámicamente, dado el mecanismo del method lookup (parte "mecánica").

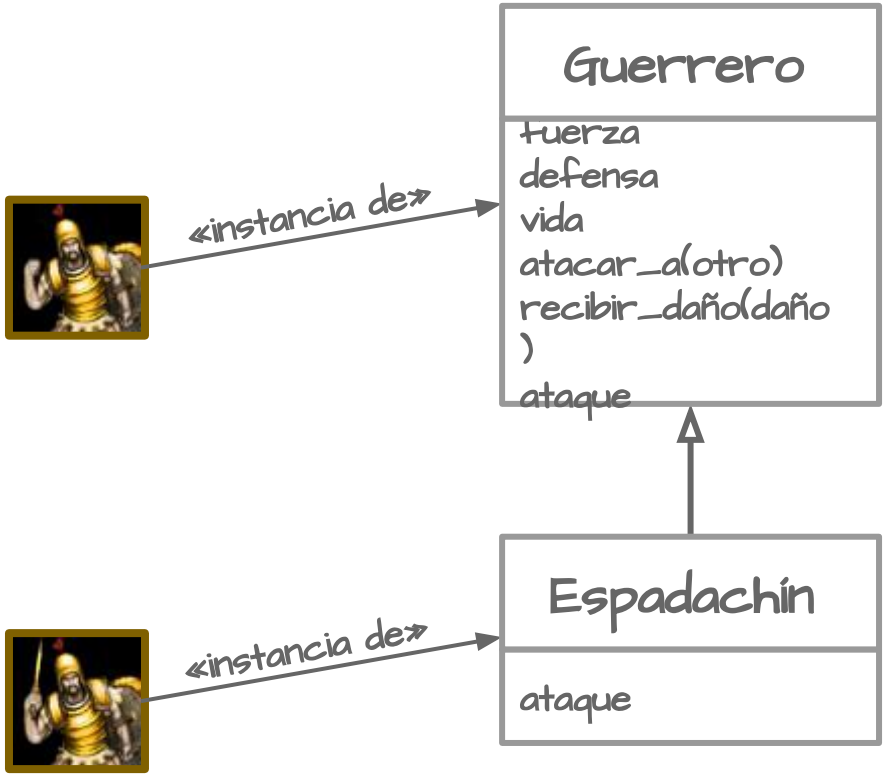
- ⚠️ Representa una teoría del conocimiento de una parte del dominio del problema. Sólo subclasificamos si es válido en el dominio.

Heurística: "todo X *es un* Y" (en tanto y en cuanto se comporta como tal, es decir, respeta el protocolo).

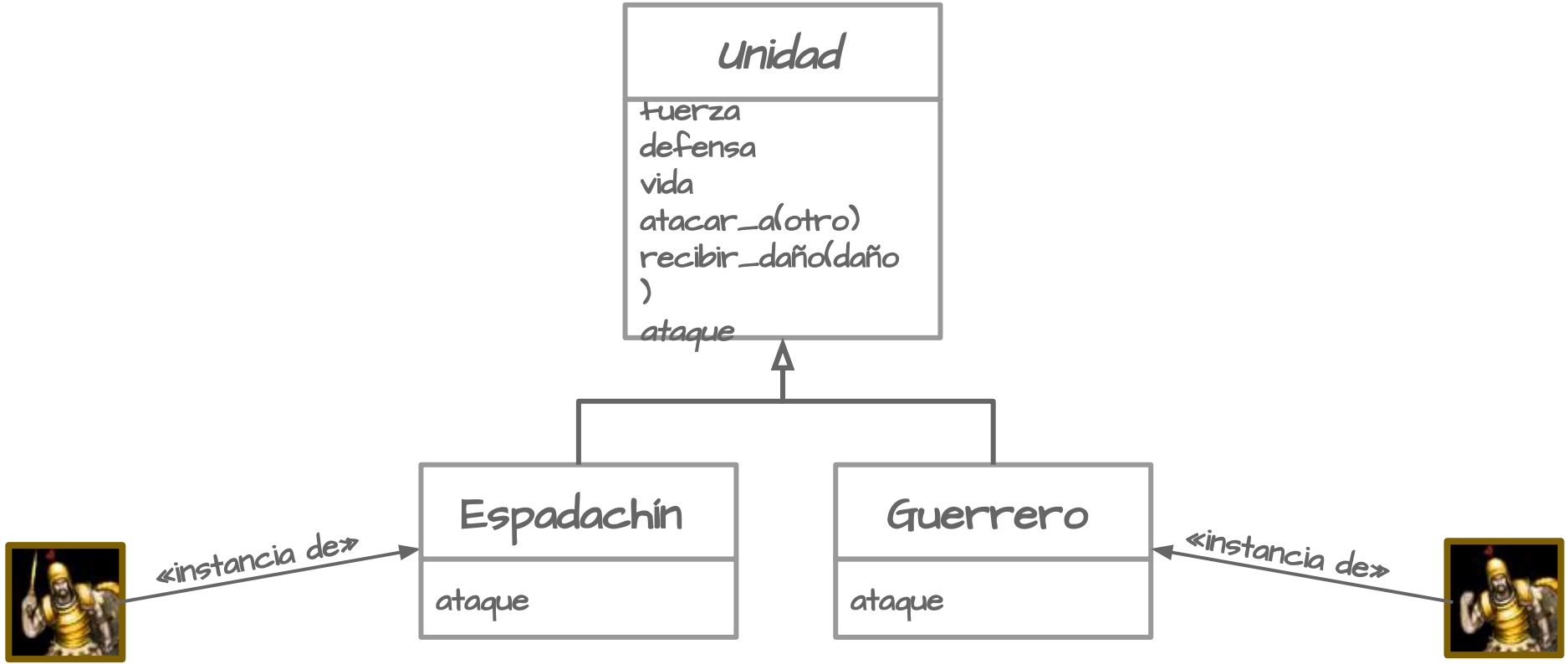


Fuente:
The many faces of inheritance: A taxonomy of taxonomy, por *Bertrand Meyer*

Ahora sí ✨


Guerrero
fuerza
defensa
vida
atacar_a(otro)
recibir_daño(daño
)
ataque
«instancia de»
Espadachín
ataque
«instancia de»

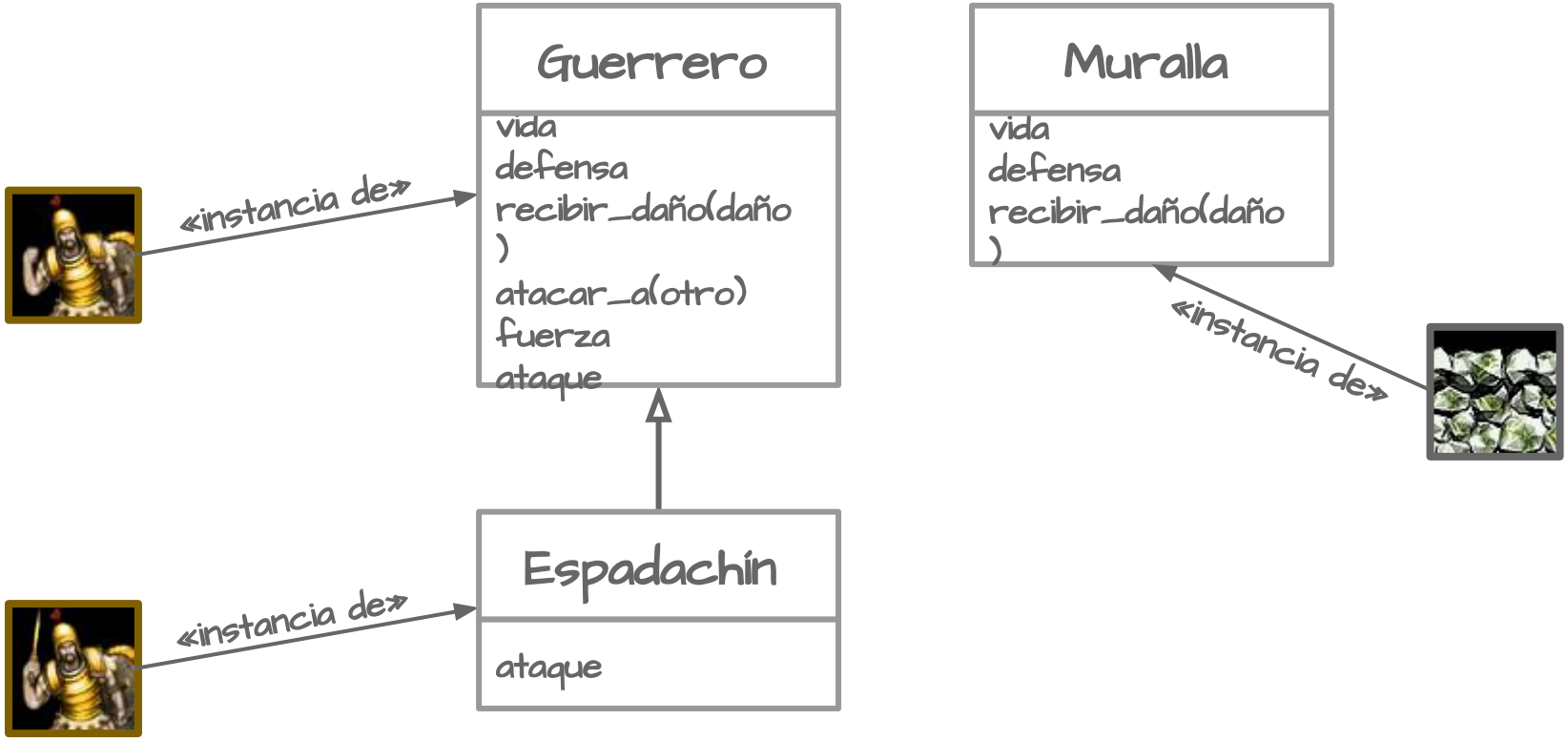
Una solución alternativa 🔮



Murallas 🧱

Una muralla es una unidad que sirve de defensa. Pueden recibir daño, pero no atacar.

La solución hasta ahora ✨

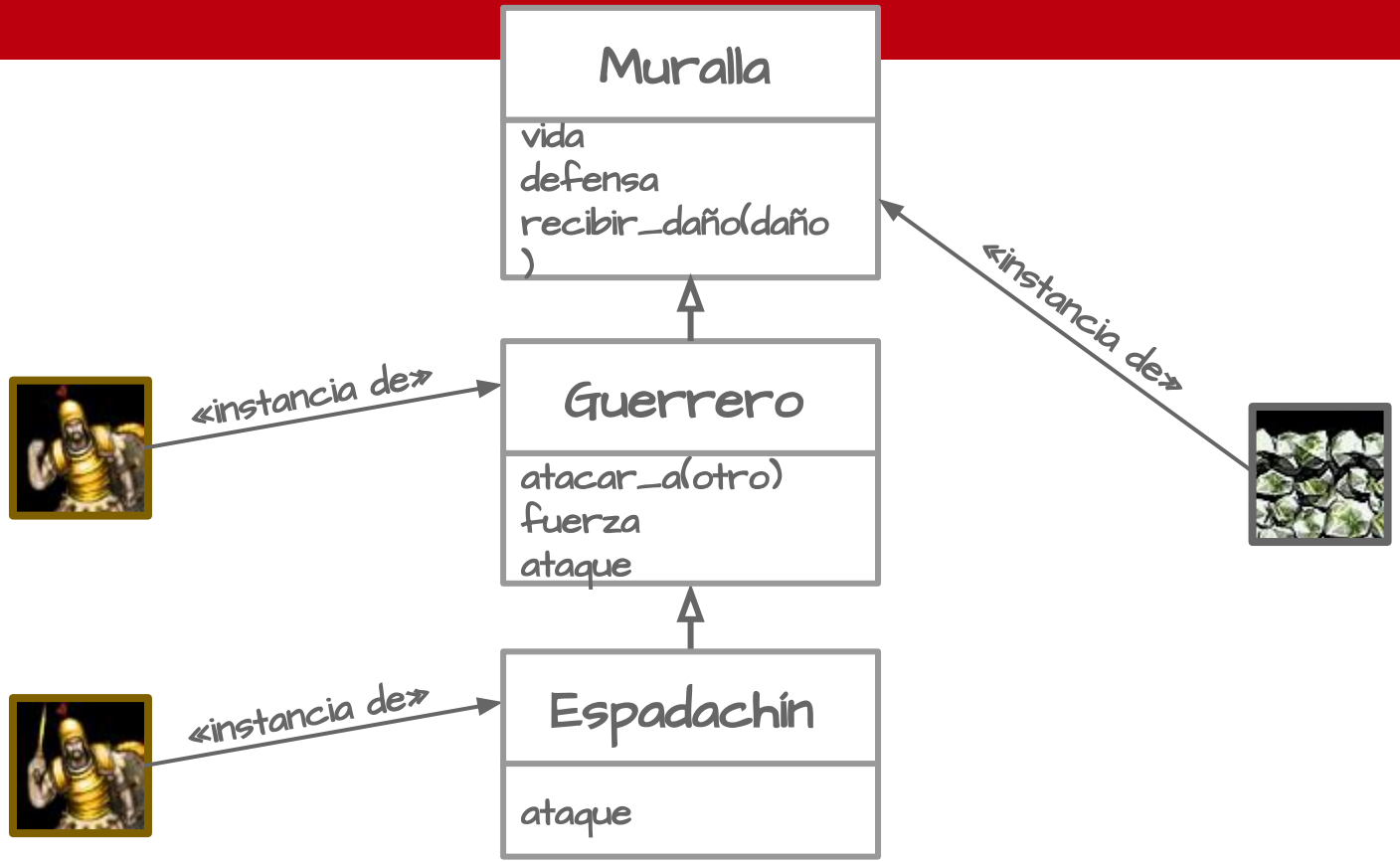


Revisemos el problema 🔍

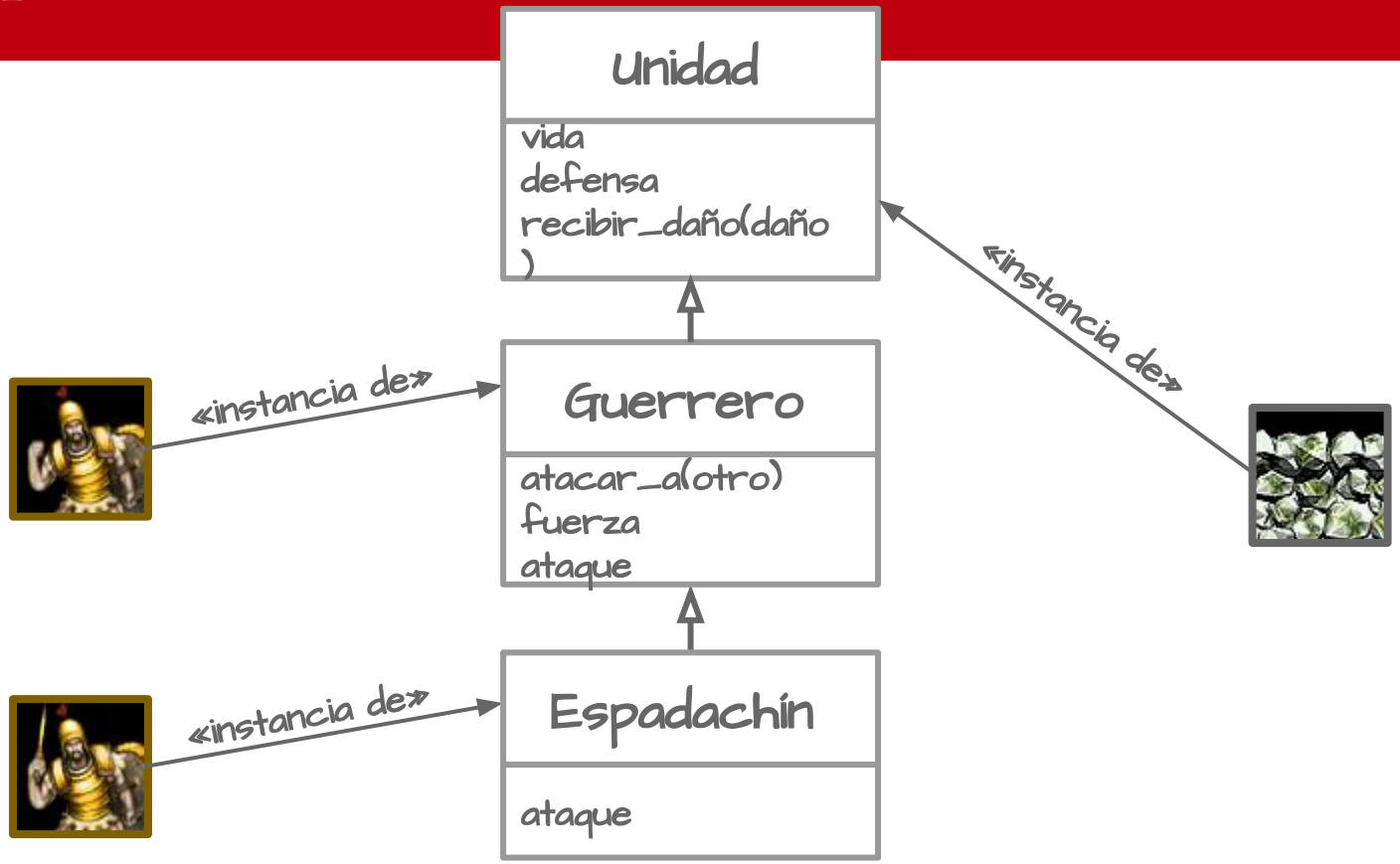
Vs.

- ✅ vida
- ✅ defensa
- ✅ recibir_daño(daño)
- ❌ atacar_a(otro)
- ❌ fuerza
- ❌ ataque

Una posible solución



Otra posible solución



Con una clase para las murallas

Fantasmas 👻

Los fantasmas son unidades que no pueden ser atacadas (porque nadie los ve), pero pueden atacar a otras unidades. Todos los fantasmas tienen 100 de ataque.

La solución hasta ahora

Fantasma

atacar_a(otro)
ataque

Guerrero

atacar_a(otro)
ataque
fuerza

Espadachín

ataque

Unidad

vida
defensa
recibir_daño(daño
)

Muralla

La solución hasta ahora

Renombrando Unidad a Defensor

Fantasma

atacar_a(otro)
ataque

Guerrero

atacar_a(otro)
ataque
fuerza

Espadachín

ataque

Defensor

vida
defensa
recibir_daño(daño
)

Muralla

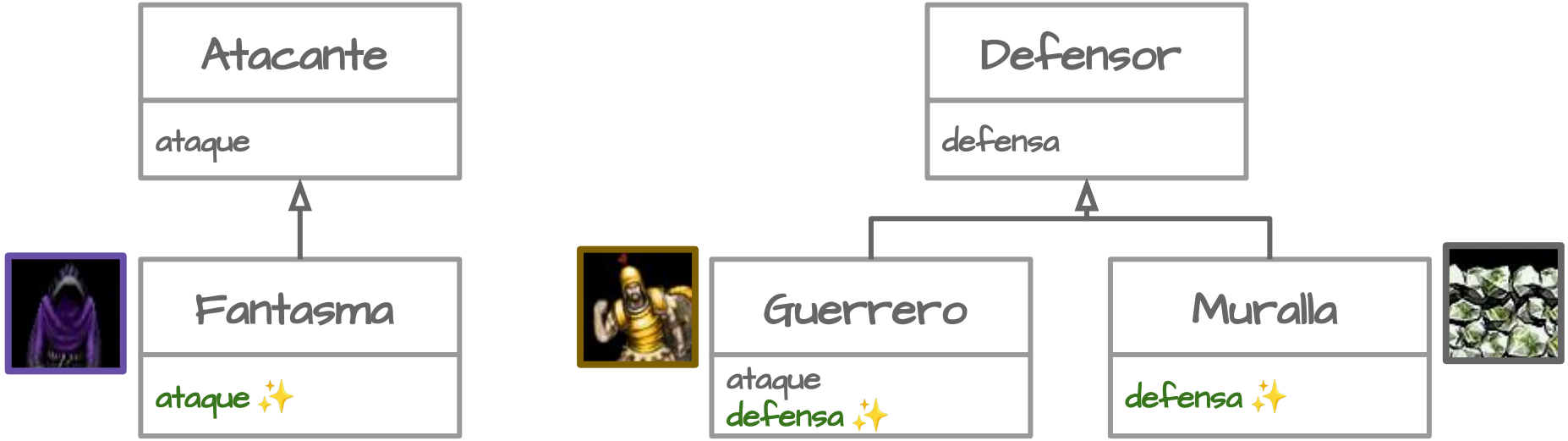
Introduciendo el concepto de Atacante

¿Cómo podemos solucionar este problema? 🤔

Lo que tenemos hasta ahora

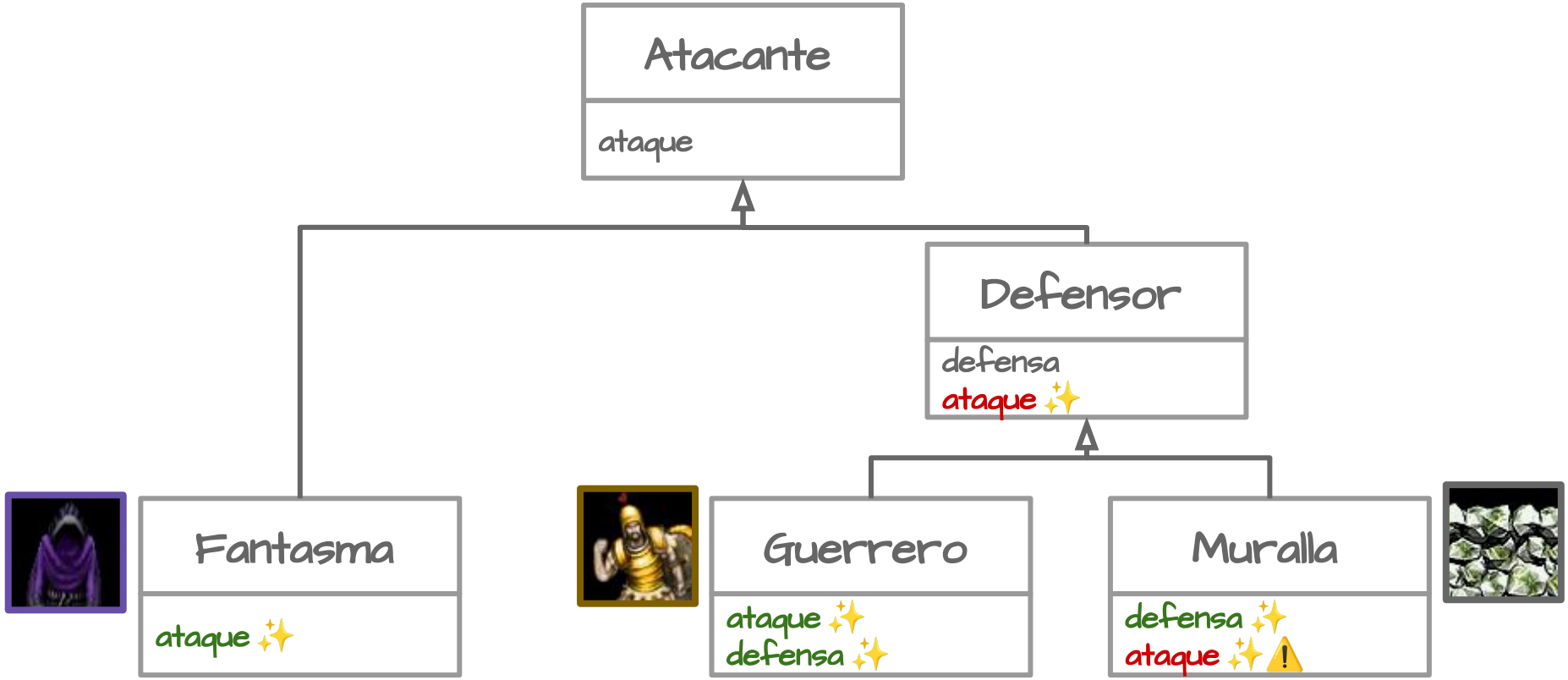
Mostremos sólo las cosas que nos interesan

Mostremos sólo las cosas que nos interesan

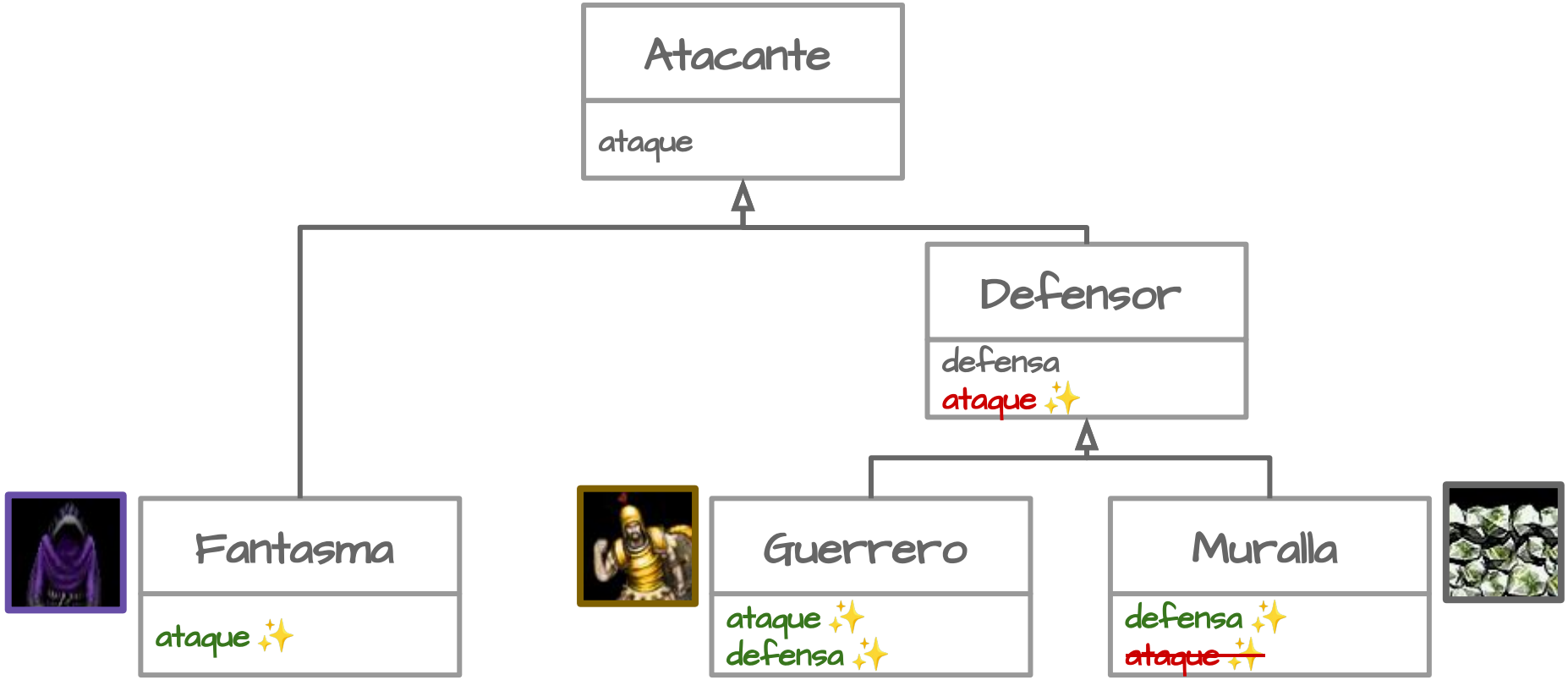


Jerarquía Forzada

Jerarquía forzada



Jerarquía forzada



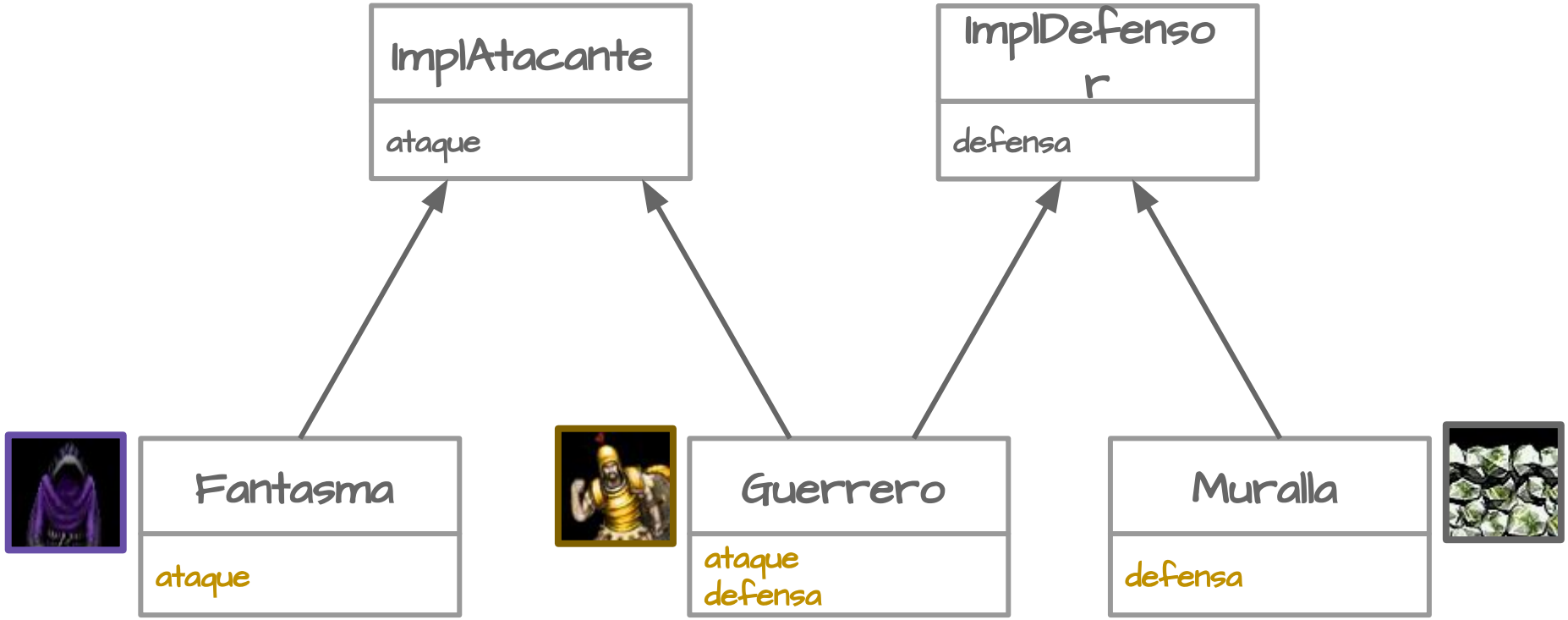
Anti-clases

Anti-clases

Unidad
ataque
defensa
SoloAtacante
ataque ✨
~~defensa~~ ✨
SoloDefensor
defensa ✨
~~ataque~~ ✨
Fantasma
ataque ✨
Guerrero
ataque ✨
defensa ✨
Muralla
defensa ✨

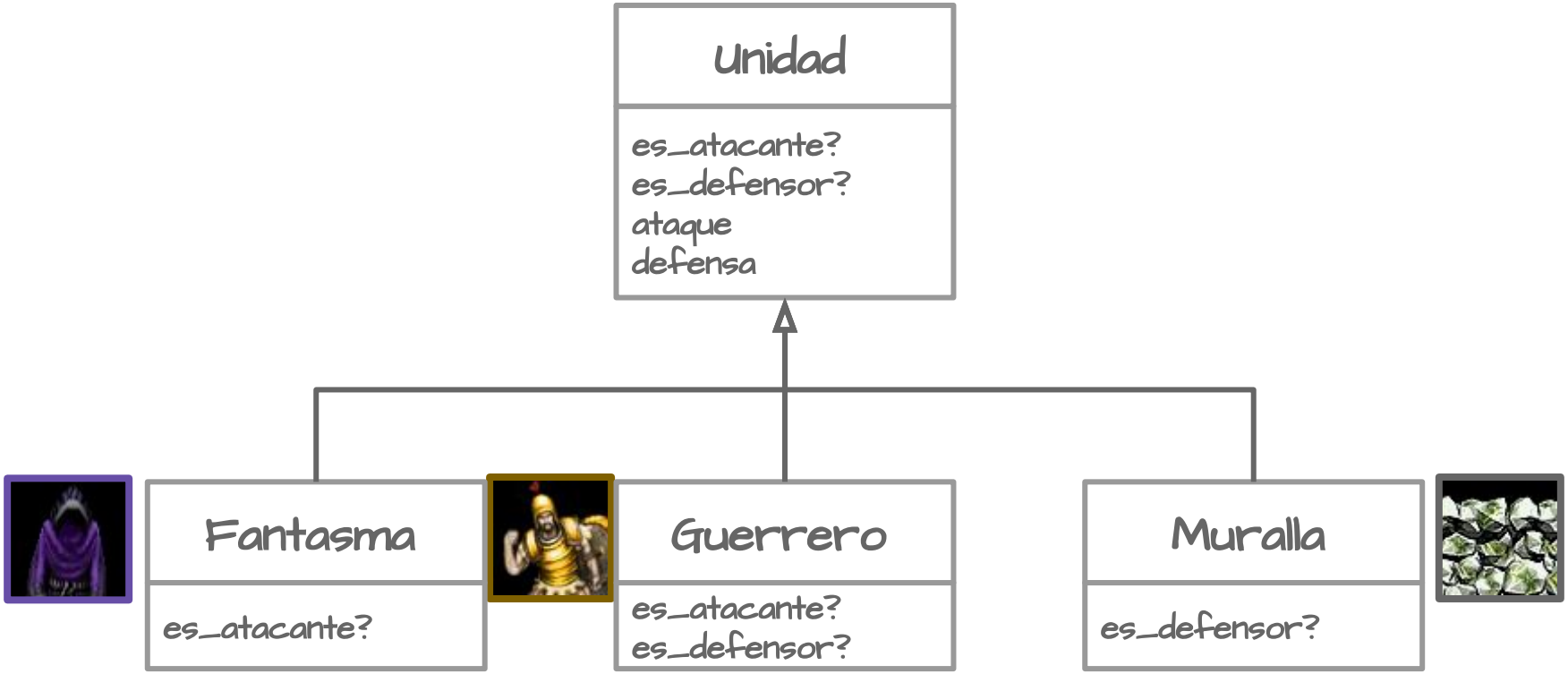
Delegación

Delegación


ImplAtacante
ataque
ImplDefensor
defensa
Fantasma
ataque
Guerrero
ataque
defensa
Muralla
defensa

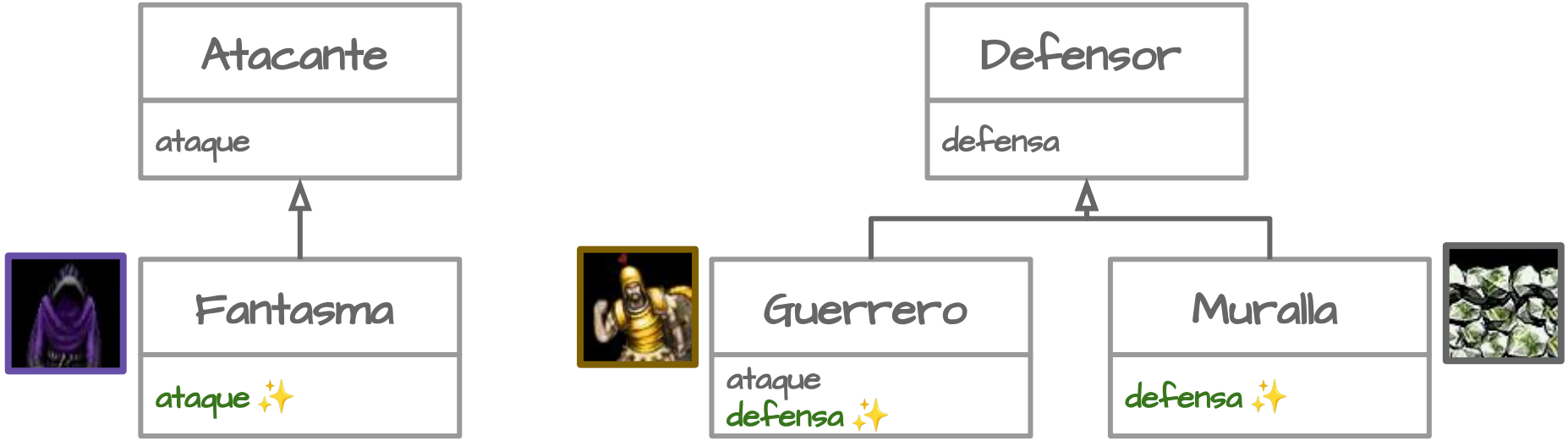
Feature flags ⛳

Feature flags



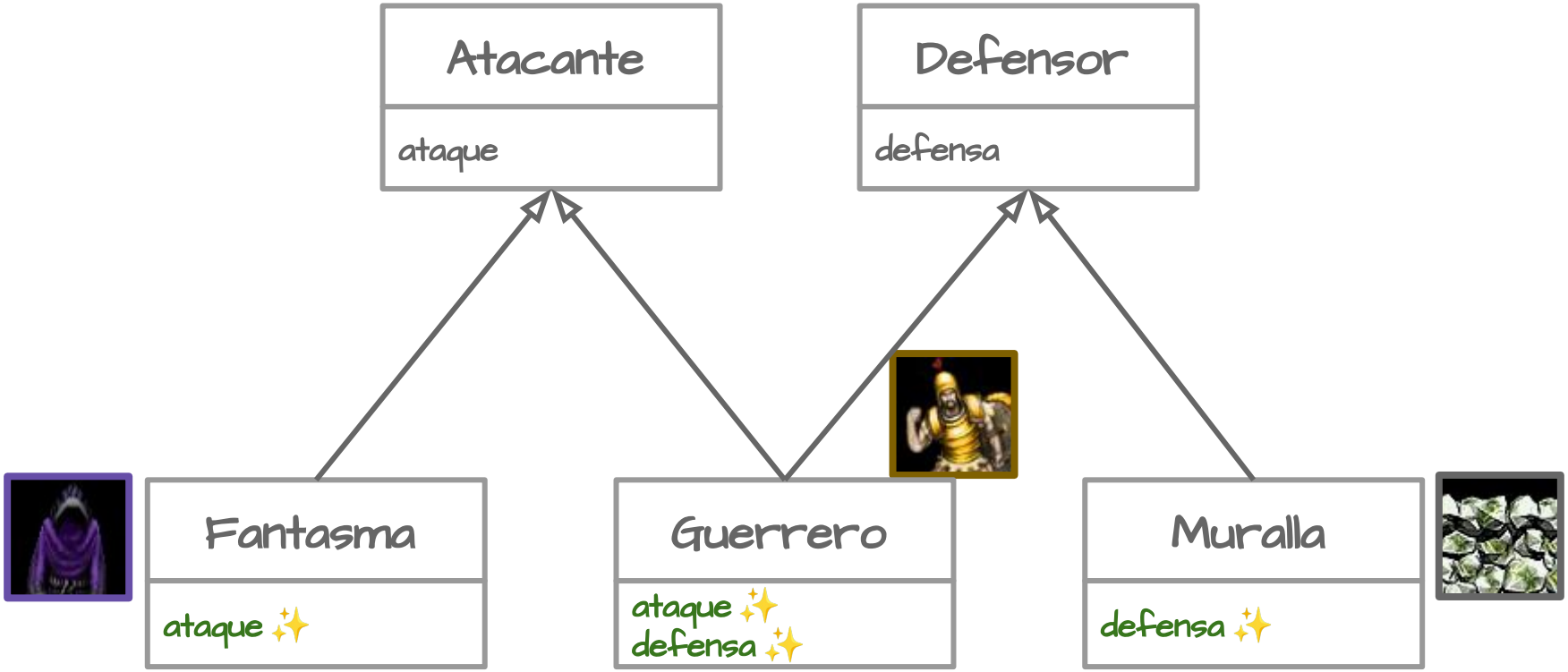
Repetir código

Repetir código

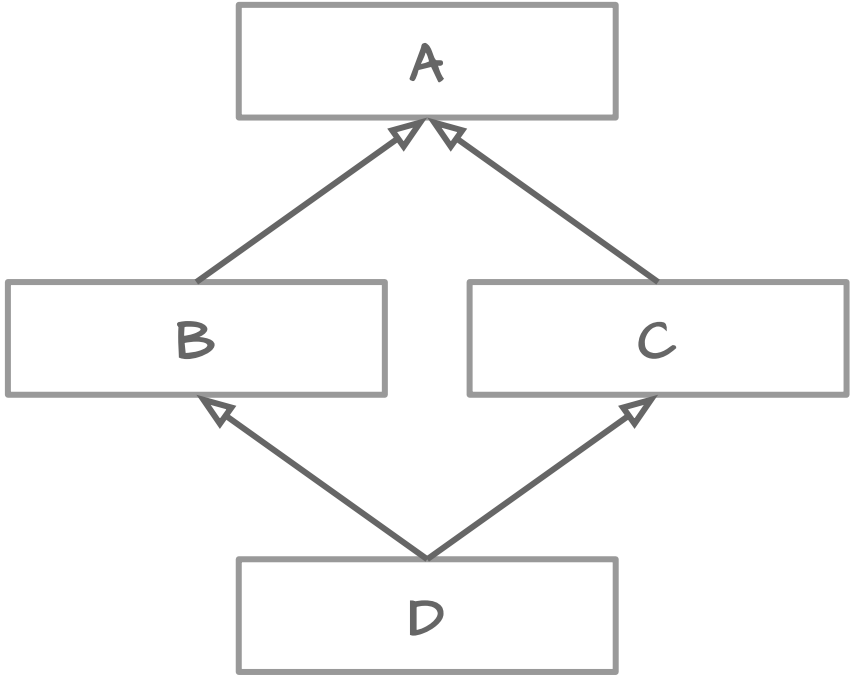


Uff!

Solución con herencia múltiple



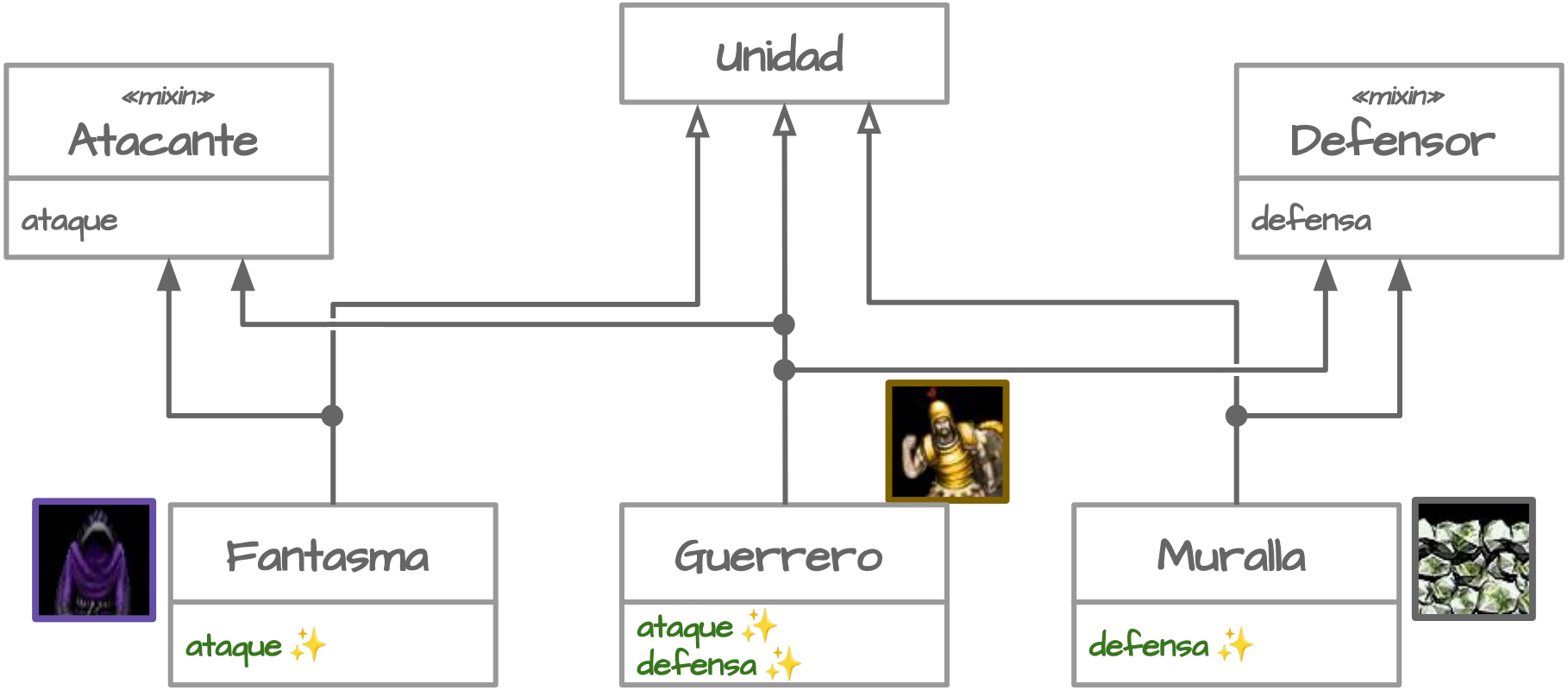
El “problema del diamante” (o *Deadly Diamond of Death*)



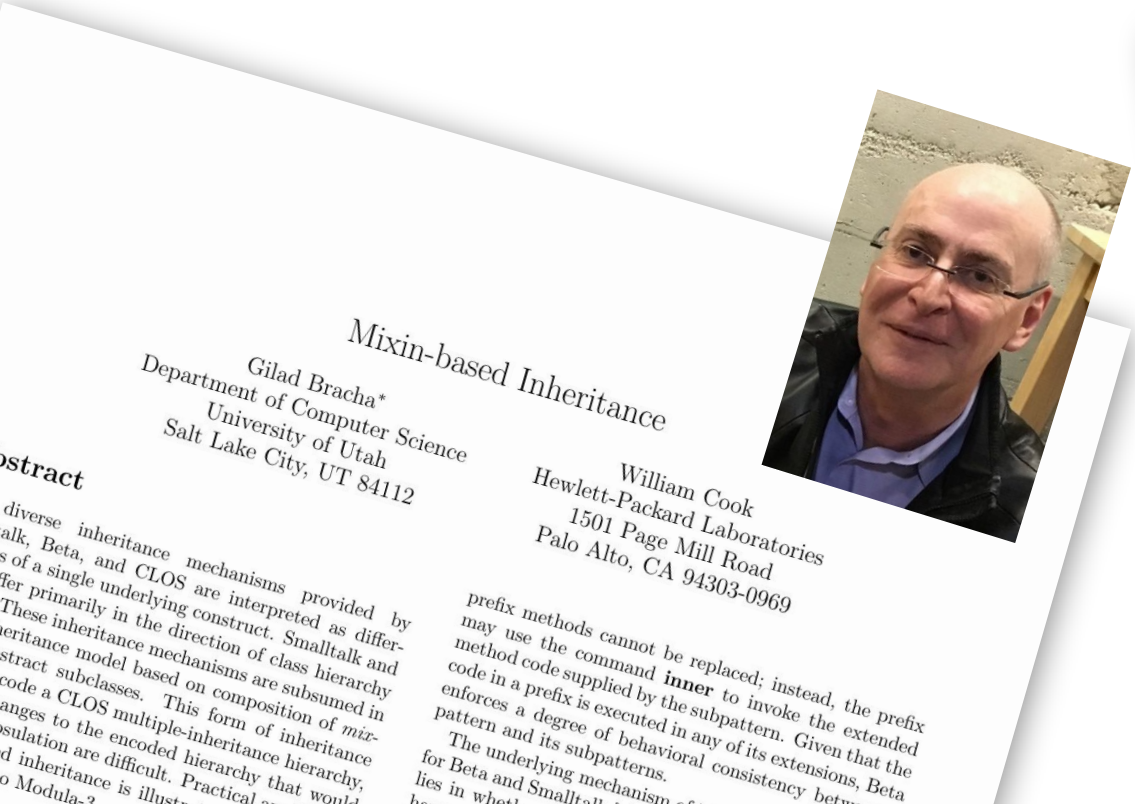
¿Qué otras alternativas hay?

Vamos al IDE

Solución con mixines



Mixines



Mixin-based Inheritance

Gilad Bracha*
Department of Computer Science
University of Utah
Salt Lake City, UT 84112

William Cook
Hewlett-Packard Laboratories
1501 Page Mill Road
Palo Alto, CA 94303-0969

ostract

diverse inheritance mechanisms provided by
alk, Beta, and CLOS are interpreted as differ-
s of a single underlying construct. Smalltalk and
ffer primarily in the direction of class hierarchy
These inheritance mechanisms are subsumed in
heritance model based on composition of *mix-*
stract subclasses. This form of inheritance
code a CLOS multiple-inheritance hierarchy,
anges to the encoded hierarchy that would
sulation are difficult. Practical
d inheritance is illust
o Modula-3

prefix methods cannot be replaced; instead, the prefix
may use the command **inner** to invoke the extended
method code supplied by the subpattern. Given that the
code in a prefix is executed in any of its extensions, Beta
enforces a degree of behavioral consistency betw
pattern and its subpatterns.
The underlying mechanism
for Beta and Smallt
lies in whet

Programming Ruby

The Pragmatic Programmer's Guide

Contents ^

Previous <

Modules

Modules are a way of grouping together methods, classes, and constants.
Modules give you two major benefits:

1. Modules provide a namespace and prevent name clashes.
2. Modules implement the mixin facility.

Referencias imágenes

Age of Empires Series Wiki [CC-BY-SA]:

- https://ageofempires.fandom.com/wiki/Hypaspist
- https://ageofempires.fandom.com/wiki/Stone_Wall_(Age_of_Mythology)
- https://ageofempires.fandom.com/wiki/Shade

Gilad Bracha's homepage:

- https://bracha.org/Site/Home.html
    - https://bracha.org/Site/Home_files/IMG_2547.jpg

Programming Ruby:

- https://www.amazon.com/Programming-Ruby-1-9-2-0-Programmers/dp/1937785491

¿Preguntas?